How to Pick the Model: A Framework for Budget-Aware Orchestration

Anonymous Author(s)
Affiliation
Address
email

Abstract

Agent systems on verified tasks invite a deceptively simple question: which backbone model should be run? The strongest model is not always the best deployment choice, because verified progress must be bought under finite wall-clock, token, API, and evaluation budgets. ~~We formulate this problem as deployment-aware model selection.~~ We formulate this problem as deployment-aware model selection for verified agent systems. A fixed router observes a pre-deployment information state and chooses a worker or harness action; the decision is scored by a checker-indexed deployment loss rather than by routing accuracy or final checker score alone. The key estimand is paired: on the same task instances, with the same router and action menu, does richer information change the selected action and reduce net deployment loss after its own measurement and context costs are charged? We instantiate the framework on a Karpathy AutoResearch-inspired CIFAR-10 edit–verify benchmark, where a prompt-only router chooses among heterogeneous workers before costly deployment runs. ~~The protocol reports when checker-score rankings disagree with deployment-loss rankings, when pre-deployment information changes routing, and whether those changes pay for themselves.~~ The promoted two-worker study shows that first-hit speed, final improvement, log-effort, and deployment loss can rank workers differently. It also gives a negative routing result: probe signals expose the task modes, but the evaluated prompt-only router does not convert that information into positive paired deployment gain. ~~The resulting protocol evaluates not which agent is best in isolation, but when information is worth buying before deploying one.~~ The resulting protocol evaluates not which agent is best in isolation, but whether a router can buy and use information well enough to improve deployment value.

1 Introduction

Practitioners building agent systems face multiple practical challenges: ~~what model~~which model (backbone) to use, how to route between models, when to spawn sub-agents, how to engineer context between them, and so on. All these possible choices often affect performance (quality of the solution), speed (time to solution), and ~~the mere token cost~~token/API cost. The question is how to ~~pick between the different possible designs~~choose among the possible designs. The common way to answer this question is benchmark racing: take a benchmark ~~as~~such as SWE-bench [1] or MLE-bench ~~[?]~~[2] and pick the agent system with the best average end-to-end performance. An increasingly long line of work ~~, based on this "recipe",~~based on this recipe is proposing better and better ~~agentic system architecture~~agentic system architectures to achieve ~~SOTA at~~state-of-the-art performance on these tasks. ~~This article investigates:~~This article investigates

~~*whether there exist other ways to answer the question of what the most suited agent system is.*~~
whether there are other ways to answer the question of which agent system is most suitable.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

Indeed, ~~(i) a large number of tasks for which such benchmark does not exist (as developing academic research [?]). And (ii) very often, the task presents instances of varied difficulty~~there are many tasks for which such benchmarks do not exist, such as developing academic research [3]. Moreover, tasks often contain instances of varied difficulty and the practitioner may want to understand whether, ~~on the single instance~~on a single instance, a lower-cost, faster model is sufficient; ~~more tools should be given to the systems~~the system should be given more tools; or a more involved orchestration is required. As a running example for the rest of the paper, ~~the question whether the user should use on their own task Haiku *vs* Sonnet *vs* Opus in Claude Code, or Claude Code *vs* Codex,~~the question of whether a user should use Haiku, Sonnet, or Opus in Claude Code, or Claude Code rather than Codex, for their own task is not fully addressed by benchmark racing.

The central issue is that one benchmark score may ~~conflates~~conflate mechanisms that imply different engineering decisions. A system can win because its worker ~~takes~~has lower per-step time or ~~token-count. Because the selected worker is~~token cost, or because the selected worker is more competent on a solver-relevant task mode. Analogously, imagine ~~some sort of pre-deployment signal~~a pre-deployment signal (~~problem definition, the context, or an initial test~~the problem definition, context, or an initial test) reveals information about the instance ~~which~~that can be used ~~to better orchestrate or to choose what backbone to pick~~to choose a better backbone or orchestration. A system can win ~~if it can orchestrate or choose the model optimally exploiting that information or not~~by exploiting that information when choosing a model or orchestration, or it can lose by failing to exploit it—as humans implicitly do when they decide to use lower or higher reasoning effort ~~reasoning~~ or different models on their tasks.

More practically, in this article *we study the deployment problem of choosing which model or harness to run on a verifiable task.* The objective is ~~time to a required verified result~~time to reach a required verified result: among actions that may eventually succeed, the useful one is the one expected to reach the verifier threshold fastest, after charging any pre-deployment measurement used to make the choice. ~~Achille and Soatto, indeed, recently showed that for verifiable tasks the time to solution, as in their Proper Time, is the performance measure to compare them.~~ Indeed, Achille and Soatto [4] recently argued that time to solution, as formalized by their Proper Time, is a performance measure for comparing agents on verifiable tasks. ~~Building on this intuition we operationalize the difference in time-to-solution of two possible designs. We show it mathematically decomposes in 4 components which isolates the mechanisms of choice mentioned above.~~ Building on this intuition, we operationalize a certified log-effort surrogate gap between two possible designs. We show that it mathematically decomposes into four components that isolate the mechanisms of choice mentioned above. ~~The experimental protocol then shows these can be estimated and lead to improvements in speed on Karpathy's AutoResearch tasks.~~ The experimental protocol estimates the operational quantities that are directly measurable and tests, by paired selected-worker deployment, whether richer pre-deployment information improves net deployment loss after measurement costs are charged.

Research questions and contributions. We organize the paper around three questions, each paired with the contribution that answers it.

1. **Can the ~~time-to-solution~~certified log-effort surrogate gap between agent designs be decomposed into meaningful terms?** Theorem 1 gives an exact four-term identity under a packed latent-mode assumption: cost, within-mode competence, routing information, and a predeclared mode-summary divergence additively explain the improvement in expected certified log-effort.

2. **Can the decomposition ~~replace~~structure or reduce end-to-end racing for model selection?** Algorithm 1 and the pilot concentration bound in Appendix A turn the identity into a structured pilot protocol whose evaluation cost depends on the number of models and task modes.

3. **When does a lower-cost model with retries beat a stronger model?** Theorem 2 gives the single-mode crossover rule, and the multi-mode accounting shows why the answer changes across task modes.

The experiments below instantiate these claims with checked trajectories and deployment accounting.

2 ~~Problem setup and deployment estimands~~Setup and accounting

We study the problem faced by a deployment system before launching an agentic run: given a task instance and a finite menu of possible workers or harnesses, which one should be run? The answer is not necessarily the model with the best final score. A deployment action must produce checked progress under finite time, token, API, and evaluation budgets.

Definition 1 (Checked deployment task)**.** *A checked deployment task is a tuple*

$$\mathcal{T} = (\mathcal{X}, \mathcal{Y}, V, \mu),$$

where $\mathcal{X}$ is an instance space, $\mathcal{Y}$ is an artifact or solution space, V is an external checker or scoring procedure on instance–artifact pairs, and μ is the deployment distribution over instances. For a realized instance $x \in \mathcal{X}$, an agent produces artifacts $y \in \mathcal{Y}$ that are scored by $V(x, y)$.

The checker may be binary, as in unit tests or proof checking, or scalar, as in validation loss or reward. The key point is externality: progress is determined by the checker, not by the model's self-report.

Definition 2 (Actions and routers)**.** *Let $\mathcal{A}$ be a finite menu of deployable actions. An action may be a single model, a fixed model harness, a retry policy, or a tool-using agent protocol. For action $a \in \mathcal{A}$ and instance x, let*

$$\ell_{\mathrm{dep}}(a, x; \xi)$$

be the predeclared deployment loss of running action a on x, where ξ denotes deployment randomness such as sampling, system latency, or data-order randomness. The loss may include checked progress, failure, persistence above threshold, wall-clock time, token/API cost, and checker/evaluation cost.

A router R observes a permitted pre-deployment information state $W(x)$ and selects an action

$$a_R(x) = R(W(x)) \in \mathcal{A}.$$

The selected action then performs the deployment run from the original task state.

Definition 3 (Budgeted externally checked task)**.** *A ~~checked deployment task~~budgeted externally checked task is a tuple*

$$\mathcal{T} = (\mathcal{X}, \mathcal{Y}, V, B_{\mathrm{dep}}, \mu).$$

Here $\mathcal{X}$ is an instance space, $\mathcal{Y}$ is a ~~artifact or solution space~~solution space, $V : \mathcal{X} \times \mathcal{Y} \to \mathcal{R}$ is an external checker or scoring procedure on instance–artifact pairs, B_{dep} is the full deployment budget and acceptance object, and μ is the deployment distribution over instances. The object B_{dep} may include a horizon, verifier budget, token/time/cost caps, and a thresholding rule that induces a binary success event from the checker score. For a realized instance $x \in \mathcal{X}$, an agent produces artifacts $y \in \mathcal{Y}$ that are scored by $V(x, y)$. An agent succeeds on $x \sim \mu$ if it produces an accepted y before exhausting B_{dep}.

The checker may be binary, as in unit tests or proof checking, or scalar, as in validation loss or reward. The key point is externality: progress is determined by the checker, not by the model's self-report.

Definition 4 (Workers, actions, and routers)**.** *Let $\mathcal{M} = \{M_1, \ldots, M_K\}$ be a finite menu of workers, where a worker is any deployable solver or fixed agent policy that can produce a candidate solution for verification. Let $\mathcal{A}$ be a finite menu of deployable actions. An action may be a single model, a fixed model harness, a retry policy, or a tool-using agent protocol. In the simplest setting, $\mathcal{A} = \mathcal{M}$; more generally, an action bundles a worker with fixed stopping, retry, tool-use, and carryover rules. ~~The deployment loss is defined locally in this action definition.~~ The deployment loss is defined in Section 4, where it is tied to first-passage cost, audit loss, and measurement loss. A routed design is a tuple*

$$d = (\mathcal{A}, R, \kappa),$$

where R is a router that selects an action after observing permitted information and κ is the declared scalar action-cost convention used in the log-effort diagnostic. When costs are logged as resource vectors, the protocol must fix the scalar reduction before evaluation. A router R observes a permitted pre-deployment information state ~~$W(x)$~~$Z_j(x)$ and selects an action

$$a_j(x) = R(Z_j(x)) \in \mathcal{A}.$$

Equivalently, a signal condition induces the routing policy $\rho_j : x \mapsto a_j(x)$. The selected action then performs the deployment run from the original task state.

This separates routing from deployment. The router chooses what to run; the selected action performs the actual checked run. Unless explicitly declared otherwise, pre-deployment records are shown only to the router and are not passed to the selected worker.

Definition 5 (Task instances and task modes)**.** *Given $\mathcal{T} = (\mathcal{X}, \mathcal{Y}, V, B_{\mathrm{dep}}, \mu)$, a task instance is a concrete realization $X = x \in \mathcal{X}$ of that same task. It is the measurable object supplied to the deployed system; the router and worker may access only the fields made available by the protocol. A task mode is a latent variable*

$$S = \sigma(X) \in \mathcal{S} = \{1, \dots, m_S\}$$

induced by a measurable map $\sigma : \mathcal{X} \to \mathcal{S}$ that assigns each instance to one of finitely many solver-relevant variants of the task. It defines a prior $\pi_s = \mathbb{P}_\mu[S = s]$ and mode-conditional instance distributions $\mu_s = \mu(\cdot \mid S = s)$. Modes are therefore not different tasks and not worker labels. They are variants of the same task: finite subpopulations of instances that share solver-relevant structure and may favor different workers or interventions.

~~The mode variable is retained by the evaluator for accounting and diagnostic comparisons. It need not be observed by either the router or the worker. This lets the same theory cover expert-defined task-mode schemas and finite evaluator-defined modes. When an experiment uses one fixed executable setting per mode, it estimates conditional panel quantities rather than the population quantities induced by μ.~~ The evaluator retains S only for accounting: neither the router nor the worker needs to observe it. In the experiments, each mode is represented by one fixed executable setting, so the reported quantities are fixed-panel analogues rather than population-level estimates under μ.

Assumption 1 (Packed latent-mode family)**.** *Each instance $X \sim \mu$ is associated with a mode $S = \sigma(X)$ in a finite mode set $\mathcal{S}$, with prior π. Conditional on mode s, action a has success probability $p(a, s) > 0$ and scalar cost*

$$\kappa(a, s) = \mathbb{E}[C_\delta(a, X; \xi) \mid S = s] > 0$$

under the declared deployment protocol, where C_δ is the normalized cost accumulated to the first certified success or the horizon. At deployment time, the router observes a signal $Z = z$ and selects one action. The evaluator defines the mode distribution induced by that signal,

$$\pi_z(s) = \mathbb{P}[S = s \mid Z = z].$$

The router is not required to output π_z or any mode distribution. Instead, the theory uses a fixed evaluator-side mode-summary distribution $q_z \in \Delta(\mathcal{S})$, with $q_z(s) > 0$ whenever $\pi_z(s) > 0$. This summary is not a router output; it is a theory-facing diagnostic distribution specified before evaluation. Write $a_z = \rho(z)$ for the action selected after observing z. For an instance whose true mode is s, the certified effort surrogate is proportional to

$$T_{\rho,q}(s, z) \propto \frac{\kappa(a_z, s)}{p(a_z, s)\, q_z(s)}. \tag{1}$$

~~Assumption 1 is deliberately restrictive. Real task instances can vary in infinitely many ways, while the theory uses a finite state abstraction. The assumption says that, for model selection, instances can be packed into modes with stable action success probabilities. The purpose is to isolate the regime in which cost, competence, information, and the predeclared mode-summary divergence are algebraically separable. Residual and calibration diagnostics measure when this finite packing is too coarse.~~ Assumption 1 is a diagnostic abstraction, not an empirical claim that the finite schema is universally correct. It isolates the regime where cost, competence, signal information, and mode-summary mismatch are algebraically separable; residual, calibration, and held-out deployment diagnostics test whether this abstraction is useful for the declared panel.

Definition 6 (Cost-adjusted certified log-effort diagnostic objective)**.** *For a fixed routed policy $\rho : Z \mapsto a_Z$ and a fixed evaluator summary q satisfying Assumption 1, define*

$$\mathcal{J}(\rho, q) = \mathbb{E}_Z \mathbb{E}_{S \sim \pi_Z}[\log \kappa(a_Z, S) - \log p(a_Z, S)] + \mathbb{E}_Z[\mathrm{H}(\pi_Z) + \mathrm{KL}(\pi_Z \| q_Z)]\,. \tag{2}$$

Lower $\mathcal{J}$ means lower expected certified log-effort under the fixed diagnostic convention. The quantity q is not optimized by the router; it is fixed by the evaluator-side diagnostic rule. When several signal conditions are compared, $\mathcal{J}$ is applied to the corresponding policy summaries; Appendix C writes this comparison explicitly for Z_j versus Z_0.

The action quantities in (2) are $\kappa(a_Z, s)$ and $p(a_Z, s)$: how costly the selected action is on mode s and how often it succeeds there. The router and information quantities are Z, π_z, and q_z: what the router is allowed to observe, what that signal reveals about the task mode, and the evaluator-side mode-summary distribution used in the KL term.

Remark 1 (Meaning of the effort surrogate)**.** *Equation* (1) *says that, on a true mode s after signal z, certified effort increases with action cost, decreases with action success probability on that mode, and increases when the evaluator-side mode summary gives too little diagnostic mass to that mode. The router is not literally routing jobs to modes; modes are evaluator-defined task variants. The quantity $q_z(s)$ is a diagnostic summary, not a deployment probability. The KL term below is therefore a mode-summary divergence under the chosen diagnostic convention. It should be interpreted as a router failure only if the experiment predefines a reproducible construction of q_z from router behavior.*

3 ~~Performance accounting~~Accounting identity

For a fixed signal value z, the routing part of the objective is the cross-entropy between the signal-induced mode distribution and the fixed evaluator-side mode-summary distribution:

$$\mathrm{CE}(\pi_z, q_z) = -\sum_{s \in \mathcal{S}} \pi_z(s) \log q_z(s) = \mathrm{H}(\pi_z) + \mathrm{KL}(\pi_z \| q_z). \tag{3}$$

The KL term is a divergence over finite task-mode distributions. It is not a token-level language-model divergence.

Theorem 1 (Four-term routed-policy decomposition)**.** *Under Assumption 1, compare a baseline policy ρ_0 that deploys fixed action a_0 without using Z and uses $q_0(\cdot \mid z) \equiv \pi$, with a deployed routed policy summarized by (ρ, q). The improvement in expected certified log-effort, $\Delta = \mathcal{J}(\rho_0, q_0) - \mathcal{J}(\rho, q)$, decomposes as*

$$\Delta = \underbrace{\mathbb{E}_Z \mathbb{E}_{S \sim \pi_Z}\left[\log \frac{\kappa(a_0, S)}{\kappa(a_Z, S)}\right]}_{\textit{cost}} + \underbrace{\mathbb{E}_Z \mathbb{E}_{S \sim \pi_Z}\left[\log \frac{p(a_Z, S)}{p(a_0, S)}\right]}_{\textit{competence}} + \underbrace{\mathrm{I}_\pi(S; Z)}_{\textit{routing information}} - \underbrace{\mathbb{E}_Z[\mathrm{KL}(\pi_Z \| q_Z)]}_{\textit{mode-summary divergence}}. \tag{4}$$

Proof sketch. Taking logs in (1) gives $\log T_{\rho,q}(s, z) = C + \log \kappa(a_z, s) - \log p(a_z, s) - \log q_z(s)$. Conditioning on $Z = z$ and averaging over $S \sim \pi_z$ converts $-\mathbb{E} \log q_z(S)$ into the cross-entropy in (3). Subtracting the prior-matched baseline cancels constants and uses $\mathrm{H}(\pi) - \mathbb{E}_Z \mathrm{H}(\pi_Z) = \mathrm{I}_\pi(S; Z)$. The full algebra is given in Appendix A. □

The four terms have different operational meanings. If the cost term is large and positive, a lower-cost action may be preferable even when it reaches success later in step count. If the competence term varies by mode, a global worker ranking is likely to hide comparative advantage. If information gain is high, the pre-deployment signal exposes solver-relevant structure. If the mode-summary divergence is high, the signal-induced mode distribution is poorly matched to the evaluator's predeclared mode summary. For two signal conditions, the identity compares the routed policies induced by those signals. A richer signal is useful only if its induced action change improves the cost–success tradeoff after the cost of obtaining the signal is charged.

Operational reading. The task mode S is not an oracle label for the best worker. The evaluator fixes a mode schema before evaluation, then estimates the mode-conditional quantities $p(a, s)$ and $\kappa(a, s)$. Routing information asks whether a signal makes those modes more identifiable. Routing shift asks whether the selected action changes when the signal is enriched. Mode-summary divergence is the KL term defined by the fixed evaluator-side mode summary q. Allocation regret, used later in the empirical protocol, is a separate action-frontier diagnostic rather than an estimator of this KL term. The final decision is not made by the information term alone; it is made by paired deployment gain on held-out instances after measurement and context costs are charged. Appendix B gives the full operationalization of these diagnostics.

3.1 ~~When lower-cost retries win~~Retry crossover design rule

Theorem 2 (Single-mode retry crossover)**.** *Fix a single homogeneous mode, success threshold δ, and horizon H. Let actions h and ℓ have one-attempt first-passage success probabilities $p_h = \mathbb{P}[\tau_\delta(h, X) \le H]$ and $p_\ell = \mathbb{P}[\tau_\delta(\ell, X) \le H]$, with expected first-passage costs κ_h and κ_ℓ. Assume $0 < p_\ell < p_h < 1$ and $\kappa_\ell < \kappa_h$. Assume retries of ℓ are conditionally i.i.d. fresh attempts, each with the same one-attempt horizon H and common Bernoulli success probability p_ℓ. The lower-cost retry depth that matches the strong action's one-attempt success probability is*

$$D_{\text{cross}} = \frac{\log(1 - p_h)}{\log(1 - p_\ell)}.$$

The corresponding integer fixed-depth retry block is cost-favorable whenever

$$\lceil D_{\text{cross}} \rceil \, \kappa_\ell < \kappa_h.$$

Remark 2 (Stopping after first retry success)**.** *If retries of the lower-cost action stop after the first success and each attempted retry has unconditional expected cost κ_ℓ, the expected cost at depth D is*

$$\kappa_\ell \sum_{d=1}^{D} (1 - p_\ell)^{d-1} = \kappa_\ell \frac{1 - (1 - p_\ell)^D}{p_\ell},$$

and the decision should also account for the residual failure probability under the deployment loss. Any difference in within-attempt time-to-success is already absorbed into κ_h and κ_ℓ.

The multi-mode case is where the four-term identity matters. A lower-cost worker can be rational on one task mode and irrational on another. A router is useful only when different actions win the cost–success tradeoff on different modes and the pre-deployment information helps identify which mode applies.

4 ~~Decomposition-guided selection~~Deployment validation protocol

~~The decomposition suggests a practical protocol. Rather than evaluate every complete design as an independent arm, the builder first estimates shared quantities: mode-conditional success, cost, predeclared signal diagnostics, and action-allocation diagnostics. Then a larger finite design menu can be scored analytically, with expensive deployment evaluations reserved for finalists and for paired validation.~~ The log-effort decomposition guides diagnostics, but deployment recommendations are made only after paired validation under a predeclared loss. The empirical protocol is compact: freeze modes, losses, cost normalizers, and signal records; estimate mode-conditional worker success and first-passage cost; audit whether richer records expose modes and change allocation; then validate finalists by paired selected-worker deployment after measurement cost.

The log-effort objective $\mathcal{J}$ is a diagnostic surrogate, not the deployment loss.

> **Diagnostic versus decision loss.** $\mathcal{J}$ explains cost–competence and signal/accounting structure under a declared diagnostic convention. ℓ_{dep} is the decision-facing loss used for paired deployment validation. We use $\mathcal{J}$ to diagnose why rankings differ, and ℓ_{dep} to decide whether a routed policy actually improves deployment value.

For deployment validation we use a predeclared first-passage loss. Given a checked progress trace $G_1, \ldots, G_H$, define

$$\tau_\delta(a, x; \xi) = \inf\{t \le H : G_t(a, x; \xi) \ge \delta\}, \qquad F_\delta(a, x; \xi) = \mathbf{1}\{\tau_\delta(a, x; \xi) = \infty\},$$

and

$$\text{Occ}_{\delta,H}(a, x; \xi) = H^{-1} \sum_{t=1}^{H} \mathbf{1}\{G_t(a, x; \xi) \ge \delta\}.$$

Let $c_\delta(a, x; \xi)$ be the normalized worker-side resource vector accumulated to $\tau_\delta \wedge H$, and let

$$C_\delta(a, x; \xi) = \lambda^\top c_\delta(a, x; \xi)$$

be the scalar cost used in $\kappa(a, s)$. The primary loss is

$$\ell_{\mathrm{dep}}(a, x; \xi) = \alpha_{\mathrm{fail}} F_\delta(a, x; \xi) + C_\delta(a, x; \xi), \tag{5}$$

with $\alpha_{\mathrm{fail}} = 1$ and normalized resource weights λ fixed before evaluation. This is the loss used in $\mathcal{J}_R(j)$ and $\Delta_R(j)$ unless a different primary loss is explicitly declared. Because the benchmark records full trajectories to horizon H, we also report the full-horizon audit loss

$$\ell_{\mathrm{audit}}(a, x; \xi) = \ell_{\mathrm{dep}}(a, x; \xi) + \alpha_{\mathrm{occ}}\big(1 - \mathrm{Occ}_{\delta,H}(a, x; \xi)\big) + \alpha_{\mathrm{qual}}\psi(G_H(a, x; \xi)), \tag{6}$$

where $\alpha_{\mathrm{occ}} = \alpha_{\mathrm{qual}} = 1$ and $\psi(u) = 1 - u$ after clipping $G_H \in [0, 1]$. The audit loss checks persistence and final quality; it does not change the first-passage cost convention in Theorem 1.

Algorithm 1 (Theory-to-deployment selection protocol).

Input: Task family, checker, finite task-mode schema $\mathcal{S}$, action menu $\mathcal{A}$, admissible progressive signals $\{Z_j\}$, pilot cell count n_{cell}, deployment repeats Q_{dep}, confirmation budget.

Output: Recommended signal-conditioned deployment policy, reported accounting terms, paired deployment gain, and calibration diagnostics.

1. *Freeze the modes and loss.* Declare $\mathcal{S}$, success threshold δ, horizon H, cost normalizers, deployment loss (5), measurement costs, and router output contract.
2. *Structured pilot.* For each action $a \in \mathcal{A}$ and mode $s \in \mathcal{S}$, estimate $\hat{p}(a, s)$, $\hat{\kappa}(a, s)$, first-hit curves, occupancy, and uncertainty intervals.
3. *Signal and router diagnostics.* For each progressive signal Z_j, report any predeclared signal-information diagnostic and log the action allocation induced by the router.
4. *Accounting score.* Compute the four log-effort terms from Theorem 1 only for quantities whose diagnostic estimators were predeclared; otherwise report allocation regret separately.
5. *Paired deployment validation.* On the same holdout instances, run the selected action under Z_0 and richer Z_j conditions, score repeated selected-worker trajectories by (5), and subtract measurement cost.
6. *Decision.* Choose the signal condition with the largest held-out $\widehat{\Delta}_R(j)$ subject to predeclared gates; use $\mathcal{J}$, frontier scores, and allocation regret as diagnostics, not as the final deployment objective.

4.1 Progressive signals and paired router value

Let $Z_j(X; \zeta_j)$ be the router-visible pre-deployment record under signal condition j, where ζ_j denotes measurement randomness. Let Z_0 be the static signal observed before choosing a deployment action. For any admissible richer condition j, the router observes Z_j, pays the corresponding measurement and router-context cost, and selects an action

$$a_R^j(X) = R(Z_j(X; \zeta_j); U_R) \in \mathcal{A}.$$

Here U_R denotes router-side randomness; for deterministic routers it is fixed and suppressed. Thus the experimental comparison is between routing policies ρ_0, ρ_j induced by different signals for the same orchestrator R, not between different orchestrators. The normalized measurement loss is

$$\ell_{\mathrm{meas}}(j, x; \zeta_j) = \lambda_{\mathrm{meas}}^\top d_j(x; \zeta_j),$$

where d_j contains measurement wall-clock, tokens, checker calls, context expansion, parsing, retry, and latency costs. Set $\ell_{\mathrm{meas}}(0, x) = 0$. The weights λ_{meas} use the same predeclared normalizers as deployment resource cost unless a separate router-cost scale is declared. In the main protocol, pre-deployment probes are outside the selected action's deployment run: they are charged through ℓ_{meas} and do not reduce the later horizon H or verifier budget B_V.

The selected-action deployment objective is

$$\mathcal{J}_R(j) := \mathbb{E}\left[\ell_{\mathrm{meas}}(j, X; \zeta_j) + \ell_{\mathrm{dep}}(a_R^j(X), X; \xi)\right]. \tag{7}$$

The expectation averages over task instances, measurement randomness, router randomness, and deployment randomness unless a conditional estimand is explicitly declared. The router-facing value of signal condition j is the paired gain

$$\Delta_R(j) := \mathcal{J}_R(0) - \mathcal{J}_R(j) = \mathbb{E}\Big[\ell_{\mathrm{dep}}(a_R^0(X), X) - \ell_{\mathrm{dep}}(a_R^j(X), X) - \ell_{\mathrm{meas}}(j, X)\Big]. \tag{8}$$

Positive $\Delta_R(j)$ means that the richer signal improves net deployment loss for the same router on the same task distribution. Allocation shift alone is insufficient:

$$\mathrm{Shift}_R(j) = \Pr[a_R^j(X) \neq a_R^0(X)]$$

measures decision relevance, not benefit.

Proposition 1 (Unbiased paired router-gain estimator)**.** *Assume task instances are sampled i.i.d. from μ, the orchestrator, action menu, and signal protocols are fixed before evaluation, measurement costs are recorded without bias, and deployment seeds are independent conditional on selected action and instance. The router is run once per instance and signal condition; the Q_{dep} deployment repeats estimate the conditional expected loss of the selected action. For condition j, estimate*

$$\widehat{\Delta}_R(j) = \frac{1}{N}\sum_{i=1}^{N}\left[\frac{1}{Q_{\mathrm{dep}}}\sum_{q=1}^{Q_{\mathrm{dep}}}\ell_{\mathrm{dep}}(a_{R,i}^0, x_i; \xi_{i,q}^0) - \frac{1}{Q_{\mathrm{dep}}}\sum_{q=1}^{Q_{\mathrm{dep}}}\ell_{\mathrm{dep}}(a_{R,i}^j, x_i; \xi_{i,q}^j) - \widehat{\ell}_{\mathrm{meas}}(j, x_i)\right].$$

If the repeated-run averages are unbiased for the expected deployment losses of the selected actions, then $\mathbb{E}[\widehat{\Delta}_R(j)] = \Delta_R(j)$.

When the same worker is selected under Z_0 and Z_j, using common deployment seeds preserves unbiasedness and reduces paired variance. When counterfactual outcomes for all actions are logged on the same instance, hindsight action regret can be reported as a diagnostic, but it is not the primary estimand. If the study fixes a finite panel of executable settings rather than sampling i.i.d. instances from μ, the same estimator describes that panel and should not be interpreted as a population-level deployment estimand.

5 Experiments and analysis

Setup. We instantiate the framework on a repeated-trajectory panel of fixed executable settings for CIFAR-10 edit–verify optimization. The task modes are three starting optimization variants of the same checked task, $\mathcal{S} = \{$*MLP-flat*, *compact CNN*, *micro ResNet*$\}$, with empirical weights $\hat{\pi}(s) = 1/3$. They are task modes because the dataset, checker, horizon, and success rule are fixed while the editable artifact's starting architecture and parameterization change, letting us test whether worker choice is mode-conditional rather than a single global ranking. The analyzed worker menu is $\{$`gpt_5_3_codex`, `gpt_5_4`$\}$. Each action is one worker under the common prompt and harness, so $\mathcal{A} = \mathcal{M}$ in this experiment. For each task-mode–worker cell we use 35 trajectories. The router is a deterministic prompt-only orchestrator evaluated under four nested pre-deployment records $Z_0, \ldots, Z_3$, from budget-only context to full unmodified-artifact baseline. The primary decision metric is the first-passage deployment loss in (5); full-horizon occupancy and final quality are audit metrics. Appendix D gives the frozen configuration, including horizon, checker budget, training budget, success threshold, signal contents, measurement-cost convention, progress normalization, smoothing rule, and logging schema.

Experiment 1: Worker frontier. We run every worker on every task mode before routing, estimating fixed-setting analogues of $p(a, s)$ and $\kappa(a, s)$. These estimates instantiate the worker-side cost and competence terms in Theorem 1; the smoothed log-effort estimator is defined in Appendix D.

The frontier already falsifies a single-score recommendation. Ranking by final improvement or first-hit speed makes `gpt_5_4` look dominant, but ranking by the charged deployment loss changes the recommendation. This is the empirical role of the decomposition: it exposes whether apparent performance comes from first-passage competence, cost, or mode-specific comparative advantage. Appendix E provides the threshold sensitivity, trajectory-level progress, cost, token, and router-record diagnostics behind these aggregate summaries.

Experiment 2: Signal audit. We audit whether richer pre-deployment records change the deterministic router's decision state. This is not an estimator of $\mathrm{I}(S; Z_j)$: a Shannon-information estimate would require a frozen finite representation or predictive model for the signal records. In the current diagnostic, budget-only records recover the three task-mode labels at 0.349 accuracy, while probe-containing records recover them perfectly on the diagnostic set. Thus the signals can expose mode structure; the remaining question is whether the router converts that structure into better actions.

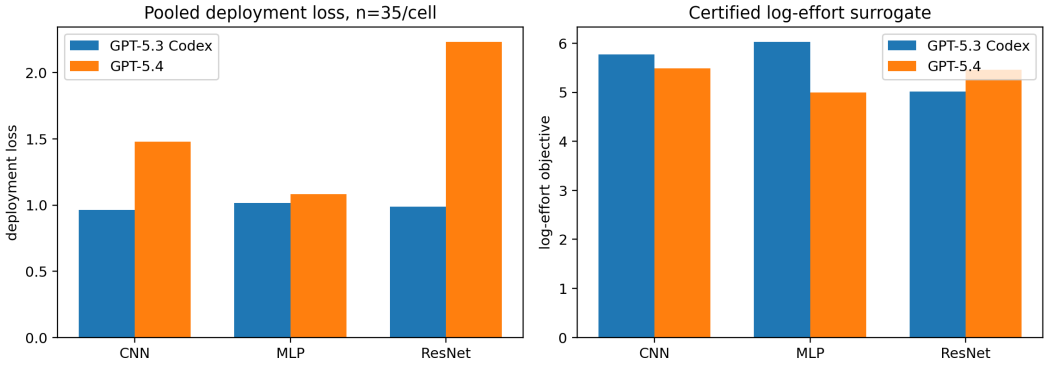

Figure 1: Two-worker frontier on the 35-run pooled panel. Left: first-passage deployment loss, lower is better. Right: certified log-effort surrogate. The two summaries disagree by mode: `gpt_5_4` reaches the threshold faster and often has higher final checked improvement, while deployment-loss accounting favors `gpt_5_3_codex` on *compact CNN* and *micro ResNet*.

Table 1: Worker-frontier summary on the 35-run pooled panel. Each cell uses 35 trajectories. Lower is better for log-effort and deployment loss.

Mode	Worker	Success	Mean τ	Log-effort	Dep. loss
MLP-flat	`gpt_5_3_codex`	32/35	7.41	6.029	1.015
MLP-flat	`gpt_5_4`	35/35	2.49	4.991	1.083
compact CNN	`gpt_5_3_codex`	34/35	6.03	5.774	0.964
compact CNN	`gpt_5_4`	35/35	3.06	5.492	1.479
micro ResNet	`gpt_5_3_codex`	35/35	2.60	5.015	0.987
micro ResNet	`gpt_5_4`	35/35	1.80	5.461	2.234

Experiment 3: Routing shift and action-frontier regret. We measure this conversion with shift rate and action-frontier regret. Shift rate is the fraction of modes for which the richer signal changes the selected worker; action-frontier regret scores the selected action against the mode-conditional frontier from Experiment 1. Both diagnostics are defined in Appendix D.

The final audit contains 480 prompt-only decisions. On real signal records, the router remains strongly biased toward `gpt_5_4`. Mean log-effort regret is 0.177 for Z_0, 0.183 for Z_1, 0.148 for Z_2, and 0.192 for Z_3. The non-deployable oracle in Figure 2 shows that the worker frontier contains exploitable mode-conditional structure, but the evaluated router does not close that gap. Additional router-record completion and signal-ablation plots are reported in Appendix E.

Experiment 4: Paired deployment gain. For each richer signal Z_j, we compare the action selected under Z_j with the action selected under Z_0 on the same executable-setting panel and subtract measurement cost:

$$\widehat{\Delta}_R(j) = \widehat{\ell}_{\mathrm{dep}}(\rho_0) - \widehat{\ell}_{\mathrm{dep}}(\rho_j) - \widehat{\ell}_{\mathrm{meas}}(j).$$

Positive values mean that the richer signal improves net deployment loss after the cost of measuring or using the signal is charged. This is the primary empirical estimand; positive signal information or lower regret alone is insufficient.

The paired validation is negative for the current prompt-only router. Against Z_0, richer real signals produce small shift rates and negative net gain after measurement cost. The bootstrap intervals from the accounting script are $[-0.116, 0.002]$ for Z_1, $[-0.127, -0.007]$ for Z_2, and $[-0.113, -0.024]$ for Z_3. This is the main falsification result: informative records are not enough; the deployed router must use them in a way that improves the selected action's checked cost–success tradeoff.

Experiment 5: Negative controls. We repeat the router and paired-gain analyses with schema-preserving controls: shuffled probe records, wrong-mode probes, and synthetic noise records

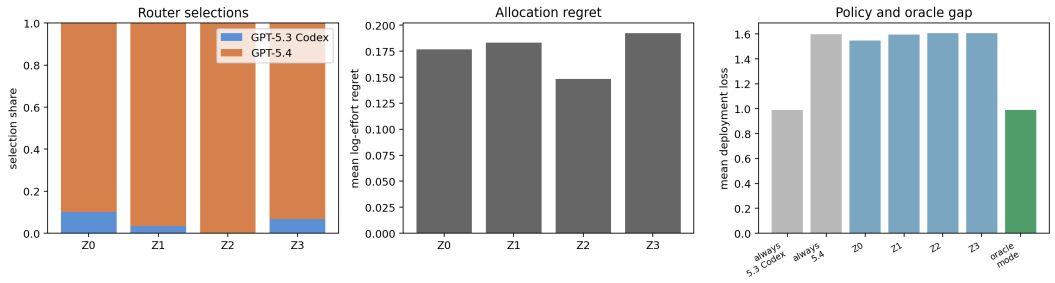

Figure 2: Two-worker router-response audit. Left: selected-worker distribution under real signal records. Middle: action-frontier regret against the 35-run pooled log-effort frontier. Right: deployment loss for always-worker baselines, prompt-router policies, and the non-deployable mode oracle. Richer records expose mode information but do not reliably move the prompt-only router toward the cost-adjusted frontier.

Table 2: Paired router validation on the final two-worker menu. Net gain is $\widehat{\ell}_{\mathrm{dep}}(\rho_0) - \widehat{\ell}_{\mathrm{dep}}(\rho_j) - \widehat{\ell}_{\mathrm{meas}}(j)$; positive is better.

Signal	Pairs	Shift	Gross gain	Meas. loss	Net gain
Z_1	30	0.133	-0.049	0.000	-0.049
Z_2	30	0.100	-0.051	0.007	-0.059
Z_3	30	0.100	-0.032	0.028	-0.060

with matched field names and approximate context burden. Controls are charged under the same measurement/context-cost convention as the corresponding real signal. They test whether router response and paired gain are specific to task-relevant signal content.

The controls block a strong routing-information claim. Wrong-mode Z_2 has a small positive mean net gain with an interval crossing zero, while several shuffled or synthetic controls are negative with intervals similar to the real signals. The correct conclusion is diagnostic rather than prescriptive: the worker frontier contains mode-conditional structure, and probes reveal that structure, but this router does not reliably turn the information into better deployment decisions. Appendix D states the full promotion rule used to separate diagnostic improvements from a deployable routing recommendation.

6 Related work

The closest classical ancestor is algorithm selection. Since Rice's formulation, solver portfolios have treated performance as instance-dependent: SATzilla and AutoFolio learn to select solvers from instance features rather than report one globally best solver [5–8]. Our contribution is the specialization to externally checked LLM agents, where a router observes a controlled pre-deployment record and is scored by paired deployment loss rather than standalone routing accuracy. Unlike AutoML systems such as auto-sklearn, which tune pipelines offline for a dataset, our unit is a deployable action for a concrete verifier-backed instance [9].

LLM routing and cascades provide the model-menu context. FrugalGPT, RouteLLM, RouterBench, LLMRouterBench, and compound-system routers study cost-quality routing over heterogeneous model pools [10–16]. We ask how much checked, task-specific information is worth before committing to an expensive agentic deployment.

Checked agent benchmarks provide the task substrate. SWE-bench evaluates repository repair against tests, AutoResearch evaluates training-loop edits against validation feedback, and theorem-proving systems use analogous external validators [1, 17]. Repeated sampling and inference-time scaling further show that lower-cost or smaller models can become competitive when verification is available [18–21]. Our question is upstream of the launched worker: which action should be selected, what should the router observe, and how should that observation be charged?

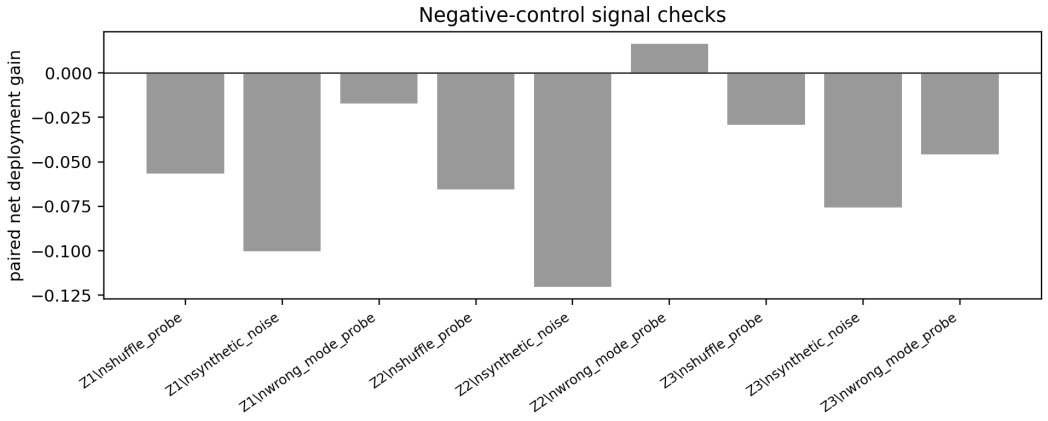

Figure 3: Negative-control paired gains for shuffled probes, wrong-mode probes, and synthetic noise records. The real records do not cleanly outperform the controls, indicating that the current prompt-only router is not reliably exploiting task-relevant measurement content.

7 Discussion and limitations

The paper has two distinct claims: an exact theory-facing accounting identity for a certified log-effort surrogate, and a deployment-facing empirical claim that must pass paired-loss validation. There are two levels of claim. The theory-facing claim is an accounting identity under Assumption 1. It is exact, but only for the certified log-effort surrogate. The deployment-facing claim is empirical: the accounting should predict or explain useful action selection under a richer deployment loss. These two claims should not be collapsed. The current experiments support the central diagnostic claim: raw worker rankings, first-passage rankings, log-effort rankings, and deployment-loss rankings can disagree on the same checked task family. That disagreement is precisely the regime in which a budget-aware orchestration framework is needed. They do not support the stronger claim that the current prompt-only router improves deployment loss. Instead, the router validation is a useful negative result: task-relevant information is measurable, but a router that mostly defaults to the stronger model can still lose after measurement cost and worker-side costs are charged.

The packed-mode assumption is the main modeling limitation. Real agent trajectories are stateful, attempts are not independent, and task modes may be fuzzy. The paper therefore reports first-passage curves, occupancy, hazard diagnostics, cost sensitivity, and surrogate-versus-loss ranking agreement instead of assuming that the finite-mode surrogate is correct. If these diagnostics are unstable, the decomposition remains a useful diagnostic language but not a reliable selection rule.

The operational task-mode schema is also a limitation. If S is too coarse, mode-conditional competence is washed out and the router appears useless. If S is too fine, sample sizes per cell collapse and the signal–mode estimates become unstable. The present evaluation fixes the executable settings and mode schema to isolate agentic trajectory variance; task-instance and schema perturbations are left as robustness extensions rather than promoted experiments.

Finally, measurement value is router-specific. A baseline probe can contain information about the best action and still fail to improve deployment loss if the actual router does not use it well or if the measurement is too expensive. For this reason, the paired gain $\Delta_R(j)$ is the main deployment estimand, while mutual-information diagnostics, mode-summary divergence, and allocation regret help explain why that gain appears or fails to appear. This distinction is the main empirical lesson of the final two-worker study. The Z_2 and Z_3 records contain task-mode information, but the induced policies have negative paired gain. The decomposition therefore functions as a claim discipline: it prevents us from promoting information availability as deployment value.

Future work. The immediate experimental extension is to add a stable lower-cost action with matched trajectory support. That would make the worker menu span a broader cost–capability frontier than the two-worker comparison reported here. It would also test the retry-crossover theorem in its intended regime, where a lower-cost worker may compensate for lower one-attempt success

through repeated checked attempts. Beyond this benchmark, the same protocol should be tested on repository repair, theorem checking, and multi-file research-code editing, where the mode schema is less artificial and measurement costs are more heterogeneous.

Pier: qui in limitations very extensive! Dobbiamo pensarle tutte. Dire perché, argue that also related work has them. And then in future direction say you want to understand if you could work them around.

Conclusion. Model choice inside an agent harness is not a scalar leaderboard problem. It is a measurable decomposition problem validated by paired deployment outcomes. The right worker depends on task state, checked competence, routing information, allocation quality, first-passage time, retry depth, and worker-side cost. Our CIFAR-10 edit–verify campaign provides the first empirical slice of that argument: the same two mature workers can trade off speed, final improvement, log-effort, and deployment loss in different ways. The two-worker routing analysis asks the deployment-facing question directly and answers it negatively for the current prompt-only router: pre-deployment information does not move allocation reliably enough to improve paired deployment loss after its measurement cost is charged. That negative result is not a failure of the framework; it is the framework doing its job, distinguishing worker-frontier structure from a validated routing recommendation.

References

[1] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[2] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Madry. MLE-bench: Evaluating machine learning agents on machine learning engineering. *arXiv preprint arXiv:2410.07095*, 2024.

[3] Mahmoud Abdelmoneum, Pierfrancesco Beneventano, and Tomaso Poggio. PoggioAI/MSc: ML theory research with humans on the loop. Technical Report Technical Report v0, MIT, 2026.

[4] Alessandro Achille and Stefano Soatto. AI agents as universal task solvers: It's all about time. *Entropy*, 28(3):332, 2026.

[5] John R. Rice. The algorithm selection problem. In *Advances in Computers*, volume 15, pages 65–118. Elsevier, 1976.

[6] Lin Xu, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. SATzilla: Portfolio-based algorithm selection for SAT. *Journal of Artificial Intelligence Research*, 32:565–606, 2008.

[7] Marius Lindauer, Katharina Eggensperger, Matthias Feurer, Bernd Bischl, and Frank Hutter. AutoFolio: An automatically configured algorithm selector. *Journal of Artificial Intelligence Research*, 53:745–778, 2015.

[8] Lars Kotthoff. Algorithm selection for combinatorial search problems: A survey. *AI Magazine*, 35(3):48–60, 2016.

[9] Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Tobias Springenberg, Manuel Blum, and Frank Hutter. Efficient and robust automated machine learning. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

[10] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

[11] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024.

[12] Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt Keutzer, and Shriyash Kaustubh Upadhyay. RouterBench: A benchmark for multi-LLM routing system. *arXiv preprint arXiv:2403.12031*, 2024.

[13] Anonymous. LLMRouterBench: A massive benchmark and unified framework for LLM routing, 2026. Anonymous ACL submission.

[14] Lingjiao Chen et al. Optimizing model selection for compound AI systems. *arXiv preprint arXiv:2502.14815*, 2025.

[15] Yanwei Yue, Guibin Zhang, Boyang Liu, Guancheng Wan, Kun Wang, Dawei Cheng, and Yiyan Qi. MasRouter: Learning to route LLMs for multi-agent systems. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15549–15572, 2025.

[16] Matei Zaharia, Omar Khattab, Lingjiao Chen, Jared Quincy Davis, Heather Miller, Chris Potts, James Zou, Michael Carbin, Jonathan Frankle, Naveen Rao, and Ali Ghodsi. The shift from models to compound AI systems. Berkeley Artificial Intelligence Research Blog, 2024.

[17] Andrej Karpathy. autoresearch: AI agents running research on single-GPU nanochat training automatically. `https://github.com/karpathy/autoresearch`, 2026. GitHub repository.

[18] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024.

[19] Charlie Victor Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. In *International Conference on Learning Representations (ICLR)*, 2025.

[20] Yangzhen Wu, Zhiqing Sun, Shanda Li, Sean Welleck, and Yiming Yang. Inference scaling laws: An empirical analysis of compute-optimal inference for problem-solving with language models. In *International Conference on Learning Representations (ICLR)*, 2025.

[21] Jinghan Zhang, Kunhao Zheng, Yici Yan, Yuchen Ouyang, and Jun Zhu. Scaling LLM inference with optimized sample compute allocation. *arXiv preprint arXiv:2410.22480*, 2024.

A ~~Proof details~~Proofs and retry rule

A.1 Four-term identity

Under Assumption 1,

$$\log T_{\rho,q}(S,Z) = C + \log \kappa(a_Z, S) - \log p(a_Z, S) - \log q_Z(S).$$

Conditioning on $Z = z$,

$$\mathbb{E}[-\log q_z(S) \mid Z = z] = \sum_s \pi_z(s) \log \frac{1}{q_z(s)} = \mathrm{H}(\pi_z) + \mathrm{KL}(\pi_z \| q_z).$$

Therefore

$$\mathbb{E}[\log T_{\rho,q}] = C + \mathbb{E}_Z \mathbb{E}_{S\sim\pi_Z}[\log \kappa(a_Z, S) - \log p(a_Z, S)] + \mathbb{E}_Z[\mathrm{H}(\pi_Z)] + \mathbb{E}_Z[\mathrm{KL}(\pi_Z \| q_Z)].$$

For the prior-matched baseline ρ_0 that deploys fixed action a_0 and uses $q_0(\cdot \mid z) \equiv \pi$,

$$\mathbb{E}[\log T_{\rho_0,\pi}] = C + \mathbb{E}_{S\sim\pi}[\log \kappa(a_0, S) - \log p(a_0, S)] + \mathrm{H}(\pi).$$

Subtracting the deployed effort from the baseline effort yields

$$\Delta = \mathbb{E}_Z \mathbb{E}_{S\sim\pi_Z}\left[\log \frac{\kappa(a_0,S)}{\kappa(a_Z,S)}\right] + \mathbb{E}_Z \mathbb{E}_{S\sim\pi_Z}\left[\log \frac{p(a_Z,S)}{p(a_0,S)}\right] + \mathrm{H}(\pi) - \mathbb{E}_Z[\mathrm{H}(\pi_Z)] - \mathbb{E}_Z[\mathrm{KL}(\pi_Z \| q_Z)].$$

The entropy difference is $\mathrm{I}_\pi(S; Z)$, giving (4).

A.2 Pilot concentration

For binary success observations in an action-mode cell, Hoeffding gives

$$\mathbb{P}(|\hat{p}(a,s) - p(a,s)| > \alpha) \le 2\exp(-2n_0\alpha^2).$$

If all true probabilities are at least $p_{\min}$, then the log-probability error obeys $|\log\hat{p} - \log p| \le \alpha/p_{\min} + O(\alpha^2/p_{\min}^2)$ on the good event. A union bound over $|\mathcal{A}|\,|\mathcal{S}|$ cells yields the conservative pilot rule

$$n_0 = O\left(\frac{\log(|\mathcal{A}|\,|\mathcal{S}|/\delta_{\text{conf}})}{\delta_{\text{acc}}^2 p_{\min}^2}\right).$$

Wilson intervals and bootstrap intervals are reported in practice because pilot cell counts are small.

B ~~Operational diagnostics for the four terms~~Operational diagnostics

The identity above becomes useful only after each term is tied to quantities logged by the deployment protocol. The checker-facing progress process is the common object used to operationalize competence, cost, information, and allocation diagnostics. Fix a horizon H and a task-normalized checked progress process $G_t(a,x) \in [0,1]$ for action a on instance x. In the main experiment, the router acts once before deployment: after observing Z, it selects one action, and that action is used for the whole trajectory. The success and cost diagnostics below are therefore trajectory-level quantities computed from the step-wise verifier trace. For lower-is-better checker losses, one admissible normalization is

$$G_t(a,x) = \text{clip}_{[0,1]}\left(\frac{L_0(x) - L_t(a,x)}{|L_0(x)| + \varepsilon_L}\right),$$

where $L_0(x)$ is the starting-artifact loss and $L_t(a,x)$ is the checked loss convention at step t. Given a success threshold $\delta > 0$,

$$\tau_\delta(a,x) = \inf\{t \le H : G_t(a,x) \ge \delta\}, \qquad F_\delta(a,x) = \mathbf{1}\{\tau_\delta(a,x) = \infty\}.$$

We also log persistence through

$$\text{Occ}_{\delta,H}(a,x) = \frac{1}{H}\sum_{t=1}^{H}\mathbf{1}\{G_t(a,x) \ge \delta\}.$$

Competence. The action competence component $p(a,s)$ is the first-success probability of action a on task mode s:

$$p(a,s) = \mathbb{P}[\tau_\delta(a,X) \le H \mid S = s].$$

Thus G_t is step-wise, but the policy competence term uses the probability that the worker selected by the router reaches the success threshold at least once before the horizon. Cumulative progress and persistence are not substituted for $p(a,s)$ in the identity. We log

$$\bar{G}_a(s) = \mathbb{E}\left[\frac{1}{H}\sum_{t=1}^{H} G_t(a,X) \mid S = s\right]$$

and $\text{Occ}_{\delta,H}$ as secondary diagnostics, especially when first-hit success saturates or when partial progress is scientifically relevant.

Cost. The action cost component $\kappa(a,s)$ is kept separate from competence. In the main protocol it is the normalized cost of deploying action a on mode s for the same trajectory convention used to define $p(a,s)$. The routed-policy cost term averages this mode-conditional cost over the actions selected under Z. The primary convention is cost accumulated until the first certified success or the horizon, $\tau_\delta \wedge H$. We log

$$C_\delta(a,x;\xi) = \lambda^\top(\widetilde{T}^{\text{worker}}_{\tau_\delta\wedge H}, \widetilde{C}^{\text{tok}}_{\tau_\delta\wedge H})(a,x;\xi),$$

and report worker wall-clock cost as primary, with token count as a sensitivity analysis. Checker runtime is logged for audit, but it is not part of $\kappa(a,s)$ unless the checker protocol or checker-call frequency differs across actions.

Routing information. The theorem term $\mathrm{I}(S; Z)$ measures how much a router-visible signal Z separates the predeclared task modes before deployment. This does not mean that the evaluator already knows the best worker for each instance. The evaluator knows only the candidate mode schema and can estimate whether Z_j makes that schema more identifiable:

$$\mathrm{I}(S; Z_j \mid Z_0) = \mathrm{H}(S \mid Z_0) - \mathrm{H}(S \mid Z_j).$$

The quantity is estimated only when a finite signal representation or calibrated predictive model for the signal records is predeclared. Otherwise the paper reports signal summaries and router-response diagnostics instead of a Shannon mutual-information estimate. It is a descriptive information diagnostic; its decision value is established only after action outcomes show that the modes predict different cost–success profiles. The paired router experiment then checks whether higher-information signals actually change the selected action and improve first-passage deployment outcomes.

Allocation-summary mismatch and allocation regret. The theorem term $\mathbb{E}_Z \, \mathrm{KL}(\pi_Z \| q_Z)$ is defined only after the evaluator predeclares a reproducible construction of the evaluator-side mode summary q_Z. Operationally, the router returns an action, not a mode label or distribution. When no such q_Z construction is frozen, the experiment reports allocation regret as a separate action-frontier diagnostic. After deployment runs, the evaluator estimates the mode-conditional frontier

$$a^\star(s) \in \arg\min_{a \in \mathcal{A}} \{\log \kappa(a, s) - \log p(a, s)\},$$

or the statistically indistinguishable set of frontier actions. For a selected action $a = \rho_j(x)$ and mode $s = \sigma(x)$, the corresponding allocation regret diagnostic is

$$r_j(x) = \big[\log \kappa(a, s) - \log p(a, s)\big] - \min_{a' \in \mathcal{A}} \big[\log \kappa(a', s) - \log p(a', s)\big].$$

This regret is computed retrospectively on a diagnostic split and validated on holdout instances; it is not an oracle input to the router. High information with high allocation regret means that the progressive signal exposed useful mode structure, but the induced action did not exploit the empirical cost–success frontier.

After these diagnostics are estimated, the experiment still reports the decision-facing first-passage deployment loss in (5) as final validation. The full-horizon audit loss in (6) is reported separately for persistence and final-quality checks. These losses are not replacements for the theorem; they test whether designs favored or explained by the diagnostics improve the practical deployment objective.

C ~~Operational decomposition under progressive signals~~Progressive-signal decomposition

The main four-term identity compares a deployed design to a prior-matched baseline. For the V3 router experiment, it is often useful to compare two progressive signal conditions for the same router. Assume a finite task mode $S \in \mathcal{S}$ and, for each signal condition j, the conditional mode distribution induced by the realized signal record

$$\pi_j(s \mid Z_j) = \mathbb{P}[S = s \mid Z_j].$$

The fixed orchestrator R induces the routing policy $a_R^j(Z_j) = R(Z_j)$. If an evaluator-side mode summary $q_R^j(\cdot \mid Z_j) \in \Delta(\mathcal{S})$ has been predeclared, define

$$\mathcal{E}_R^j(s, Z_j) = \frac{\kappa(a_R^j(Z_j), s)}{p(a_R^j(Z_j), s)\, q_R^j(s \mid Z_j)}.$$

The log-effort gain of signal condition Z_j over Z_0 is

$$\Delta_R^{\log}(j) := \mathbb{E}[\log \mathcal{E}_R^0(S, Z_0) - \log \mathcal{E}_R^j(S, Z_j)].$$

Expanding the cross-entropy terms gives

$$\Delta_R^{\log}(j) = C_R(j) + \Phi_R(j) + G(j) + M_R(j),$$

where

$$C_R(j) = \mathbb{E}\left[\log \frac{\kappa(a_R^0(Z_0), S)}{\kappa(a_R^j(Z_j), S)}\right],$$
$$\Phi_R(j) = \mathbb{E}\left[\log \frac{p(a_R^j(Z_j), S)}{p(a_R^0(Z_0), S)}\right],$$
$$G(j) = \mathrm{H}(S \mid Z_0) - \mathrm{H}(S \mid Z_j),$$
$$M_R(j) = \mathbb{E}\Big[\mathrm{KL}(\pi_0(\cdot \mid Z_0) \| q_R^0(\cdot \mid Z_0)) - \mathrm{KL}(\pi_j(\cdot \mid Z_j) \| q_R^j(\cdot \mid Z_j))\Big].$$

When Z_j refines Z_0, $G(j) = \mathrm{I}(S; Z_j \mid Z_0)$. This signal-mode decomposition is theory-facing and excludes measurement cost unless such a term is added explicitly. It is a diagnostic companion to the deployment-gain estimand (8), not a replacement for it.

D ~~Operational estimator details~~Full selection protocol and experimental configuration

Panel of fixed executable settings. The main experiment is a repeated-trajectory panel of fixed executable settings rather than a population draw from μ. Each task mode s is represented by one executable setting

$$\chi_s = (A_s, D, \mathrm{split}, u, \eta_s, V, B_V, H, \delta),$$

where A_s is the editable `train.py` template and starting model class, D is CIFAR-10, split is the fixed train/validation split, u is the model-initialization seed, η_s are starting optimization and architecture hyperparameters, V is the checker, B_V is the checker budget, H is the edit–verify horizon, and δ is the success threshold. Panel-level aggregates use $\hat{\pi}(s) = 1/3$. The analyzed worker menu is {`gpt_5_3_codex`, `gpt_5_4`}. The paper-facing comparison sets $\mathcal{A} = \mathcal{M}$: each deployable action is a single worker run under the common prompt and harness.

Task-mode construction. The paper-facing task modes are named by the starting model family in the editable artifact. *MLP-flat* (`mlp_flat`) is a fully connected classifier over flattened CIFAR-10 inputs; *compact CNN* (`cnn_compact`) is a small convolutional classifier with local spatial inductive bias; and *micro ResNet* (`resnet_micro`) is a reduced residual convolutional classifier with skip connections. They are not different datasets or different workers. They are evaluator-defined variants of the same CIFAR-10 edit–verify task: the dataset, split, checker, prompt harness, horizon, and success threshold are fixed, while the initial architecture family and associated template parameters change. This choice makes the mode variable operationally meaningful for the theory, because different starting artifacts can change both first-passage competence $p(a, s)$ and cost $\kappa(a, s)$ for the same worker.

Progressive signal records. The router sees one of four nested pre-deployment records. Z_0 contains only the task name, checker objective, worker menu, and budget. Z_1 adds the initial `train.py` plus a static workload summary: starting model class, parameter count, optimizer and budget settings, dataset/checker identifiers, and parse/runtime availability. Z_2 adds a low-cost checker probe of the unmodified artifact at a predeclared probe budget. Z_3 adds the full unmodified-artifact baseline under the deployment checker budget. Probe records include validation loss, validation accuracy, training seconds, total seconds, optimizer steps, parameter count, parse/runtime status, and probe budget. Probe runs are outside the selected worker deployment: they are charged through ℓ_{meas} and do not consume the later deployment horizon or checker budget. The evaluator-only baseline used to compute G_t is router-visible only in Z_3.

Experiment accounting. The worker frontier uses 35 trajectories per task-mode–worker cell. The selected-router validation uses the same fixed panel and reports all richer signals $j \in \{1, 2, 3\}$ rather than selecting a single best signal on the same data. Negative controls keep the signal schema and approximate context burden of the matched real signal: shuffled probes, wrong-mode probes, and synthetic noise records are charged under the same measurement/context-cost convention as the real record they mimic. For the router audit, the prompt-only router selects `gpt_5_4` on 27/30 budget-only records, 29/30 static-workload records, 30/30 probe records, and 28/30 full-scout records.

Table 3: Frozen operational configuration for the main protocol. All values are fixed within a task mode; repeated runs vary only the agentic trajectory.

Component	Fixed value
Dataset and split	CIFAR-10, 50,000 training examples and 10,000 validation examples; balanced subset, label noise 0.0, imbalance ratio 1.0; the train/validation split is fixed in the main protocol
Data seed	Common fixed data/subset seed; alternative split seeds are reported only as robustness variants
`train.py` initialization seed	`SEED=42`; used for Python, NumPy, PyTorch, CUDA seeds, deterministic PyTorch mode, and the training-loader generator; alternative initialization seeds are robustness variants
Training transforms	random crop with padding 4, random horizontal flip, CIFAR-10 normalization; validation uses normalization only
Training budget per checker call	fixed-step mode with `A_MAX_STEPS=256`; time budget `120s` is a fallback, not the primary scoring convention
Common hyperparameters	`BATCH_SIZE=64`, `NUM_WORKERS=0`, Adam, learning rate $5 \cdot 10^{-4}$, weight decay 10^{-4}, betas $(0.9, 0.999)$, no LR schedule
Task mode	*MLP-flat* (`mlp_flat`), *compact CNN* (`cnn_compact`), or *micro ResNet* (`resnet_micro`)
Shared template parameters	depth 2, base channels 12, channel multiplier 2, batch norm enabled, dropout 0.0, and hidden width 48
Agent horizon	$H = 20$ edit–verify steps; trajectories continue to H after first success to measure persistence and final quality
Progress normalization	$\varepsilon_L = 10^{-8}$ in the denominator of G_t
Primary loss weights	$\alpha_{\text{fail}} = 1$; normalized worker wall-clock is the primary scalar C_δ; token cost is a sensitivity analysis
Audit loss weights	$\alpha_{\text{occ}} = \alpha_{\text{qual}} = 1$, $\psi(u) = 1 - u$
Measurement cost	pre-deployment probes are charged through ℓ_{meas} using the same wall-clock normalizer as C_δ; context-token cost is reported as sensitivity
Router configuration	deterministic prompt-only router with fixed model identifier, prompt hash, parser, temperature, top-p, and seed policy recorded before evaluation

Promotion rule. A routing recommendation is promoted only when worker-frontier estimates, cost sensitivity, router shift, action-frontier regret, paired gain after measurement cost, and negative controls agree. If diagnostics improve but paired deployment gain does not, the report treats the result as a mechanism failure rather than a useful router.

Trajectory record. Each run stores the worker alias, resolved model identifier, API date, split, task family, task mode, seed, maximum horizon H, signal condition, prompt hash, router model identifier, router temperature/top-p, router seed policy, parser version, router output, selected worker, and cost convention. Each step stores the prompt-visible state, proposed `train.py` artifact, parse status, checker result, validation loss, validation accuracy, relative improvement, incremental worker wall time, token usage, and checker runtime for audit. Parser failures are logged as unsuccessful routing or deployment records under the predeclared parser-failure policy.

Success and time-to-verification. For trajectory i, τ_i is the first step at which the relative improvement threshold is crossed; censored failures have $\tau_i = \infty$. For each action-mode cell,

$$\hat{F}_a(s,h) = \frac{1}{n_{a,s}} \sum_i \mathbf{1}\{\tau_i \le h\}, \qquad \hat{S}_a(s,h) = 1 - \hat{F}_a(s,h).$$

The empirical hazard is

$$\hat{\lambda}_a(s,h) = \frac{\sum_i \mathbf{1}\{\tau_i = h\}}{\sum_i \mathbf{1}\{\tau_i \ge h\}}.$$

The raw first-success estimate is $\hat{p}(a,s) = \hat{F}_a(s,H)$. For log-effort scores, the protocol uses the predeclared smoothed estimate

$$\tilde{p}(a,s) = \frac{k_{a,s} + 1/2}{n_{a,s} + 1},$$

where $k_{a,s}$ is the number of first successes in the cell. This prevents zero-success pilot cells from making log scores infinite; raw success counts are still reported.

Cost. The primary cost is cumulative worker wall-clock time to $\tau \wedge H$. Secondary worker-side cost is cumulative token count. Checker runtime is logged as a protocol audit field and excluded from action-cost comparisons unless the checker protocol differs across actions. All plots that support model recommendations must be reproducible under at least worker wall-clock and token-count conventions. When a worker fails before producing a parseable artifact, its cost is still counted and the step is marked unsuccessful. The scalar estimator matched to Assumption 1 is the cell mean

$$\hat{\kappa}(a, s) = \frac{1}{n_{a,s}} \sum_{i:S_i=s} C_{\delta,i}(a),$$

or its fixed-setting analogue $\hat{\kappa}_{\chi_s}(a)$ in Section 5.

Information, allocation summaries, and regret. For each progressive signal Z_j, a Shannon information estimate is reported only if the protocol predeclares a finite signal representation or calibrated predictive model for estimating $\pi_j(\cdot \mid Z_j)$ on held-out instances. Otherwise the report uses qualitative signal summaries and router-response diagnostics without calling them estimates of $\mathrm{I}(S; Z_j)$. The KL mode-summary divergence term is reported only if the evaluator also freezes a reproducible construction of $\hat{q}_R^j(\cdot \mid Z_{j,i})$ before seeing evaluation outcomes. For the deterministic router, the empirical panel shift rate is

$$\widehat{\mathrm{Shift}}_R(j) = \sum_{s \in \mathcal{S}} \hat{\pi}(s) \mathbf{1}\{\rho_j(\chi_s) \neq \rho_0(\chi_s)\}.$$

In the main fixed-setting protocol, the primary operational diagnostic is allocation regret,

$$\hat{r}_j(x_i) = \big[\log \hat{\kappa}(a_{j,i}, s_i) - \log \tilde{p}(a_{j,i}, s_i)\big] - \min_{a \in \mathcal{A}} \big[\log \hat{\kappa}(a, s_i) - \log \tilde{p}(a, s_i)\big],$$

where $a_{j,i} = R(Z_j(x_i))$ and $s_i = \sigma(x_i)$.

Uncertainty. Success probabilities use Wilson intervals. Aggregated objectives, paired gains, and calibration diagnostics use bootstrap or randomization tests at the sampling unit declared for the study. If the main protocol remains a panel of fixed executable settings, paired randomization tests over repeated trajectories are descriptive for that panel and should not be interpreted as population-level inference. If independent task instances are added, uncertainty is reported at the task-instance level.

E ~~Additional diagnostic plots~~Additional plots

This appendix collects auxiliary diagnostics generated from the experimental logs. The threshold, trajectory, and cost diagnostics below use the same analyzed workers, `gpt_5_3_codex` and `gpt_5_4`. These appendix plots use the same 35-trajectory-per-cell panel as the main analysis. The pooled appendix plots are robustness and mechanism diagnostics for the same accounting quantities reported in the main text.

E.1 Pooled threshold and trajectory diagnostics

E.2 Pooled cost diagnostics

Figure 4: Zoomed threshold sensitivity for the two mature workers on the 35-run pooled panel. The red line marks the operating threshold $\delta = 0.05$. At this threshold, entry success is near saturated while mean first-hit time remains low, which makes $\delta = 0.05$ a pragmatic operating point for studying time-to-certified-solution. The sharp drop at larger thresholds shows why the exact threshold is not an innocuous plotting choice.

Figure 5: Threshold sensitivity by worker on the 35-run pooled panel. `gpt_5_4` remains stronger at higher thresholds, while `gpt_5_3_codex` degrades earlier; this supports the competence term in Theorem 1.

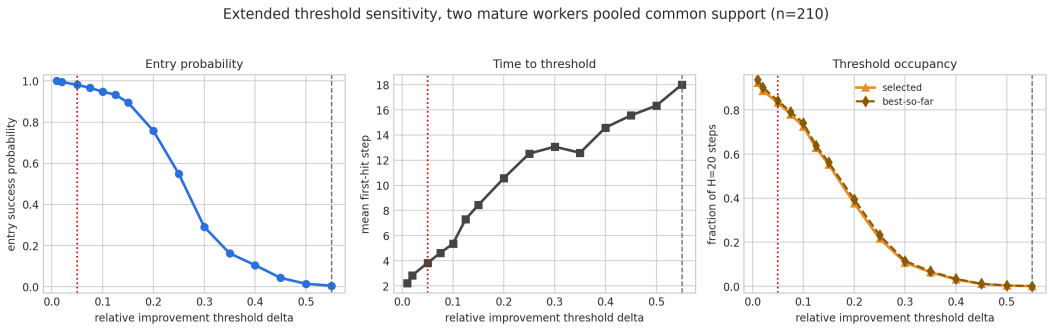

Figure 6: Extended threshold sensitivity for the two-worker pooled panel. The full threshold range shows that high-threshold success is rare even when $\delta = 0.05$ is nearly saturated. This is why the paper reports first-passage curves, occupancy, and final quality rather than relying on a single binary success rate.

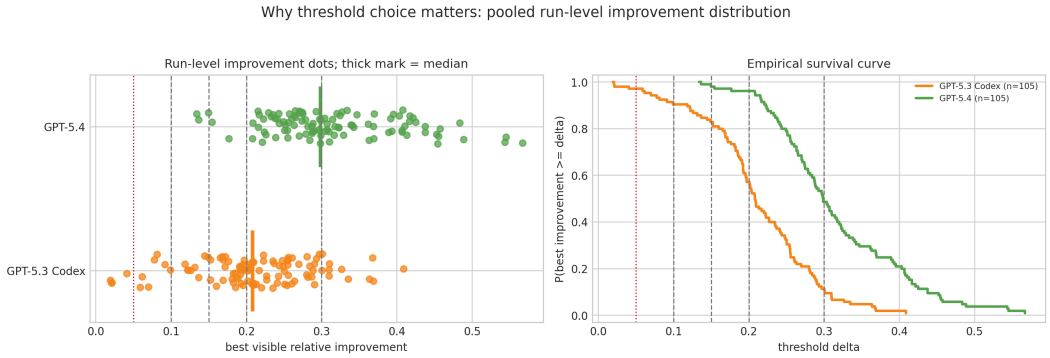

Figure 7: Run-level improvement distribution for the two mature workers. Dots show final best-visible relative improvement and thick ticks mark worker medians. The survival plot makes the threshold choice explicit: near $\delta = 0.05$, most runs succeed, but the ordering and separation change at higher thresholds. This diagnostic supports the paper's claim that final quality, first-passage success, and deployment loss are distinct empirical objects.

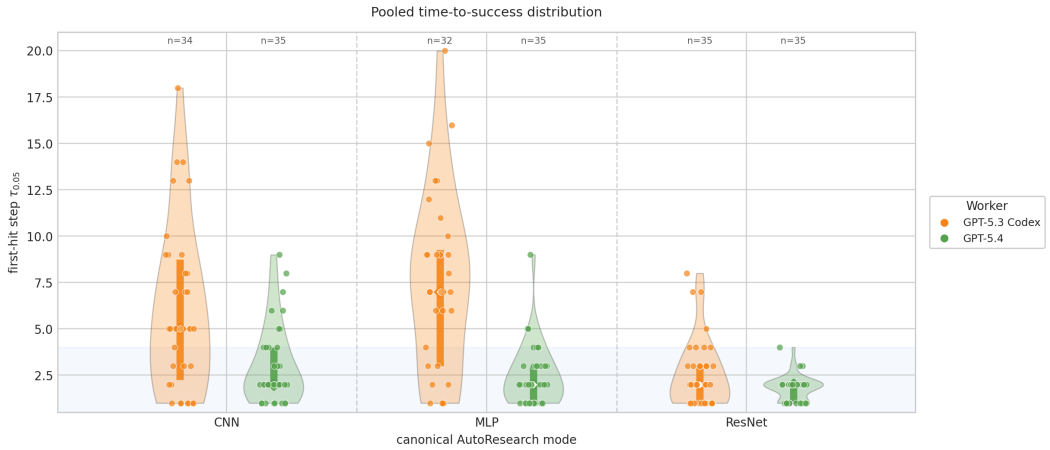

Figure 8: Pooled time-to-success distribution by task mode and worker. The violin plots expose mode-conditional heterogeneity: `gpt_5_4` typically hits the threshold quickly, while `gpt_5_3_codex` has broader first-hit times, especially on MLP. This is the empirical object behind the cost and competence terms of the four-term decomposition.

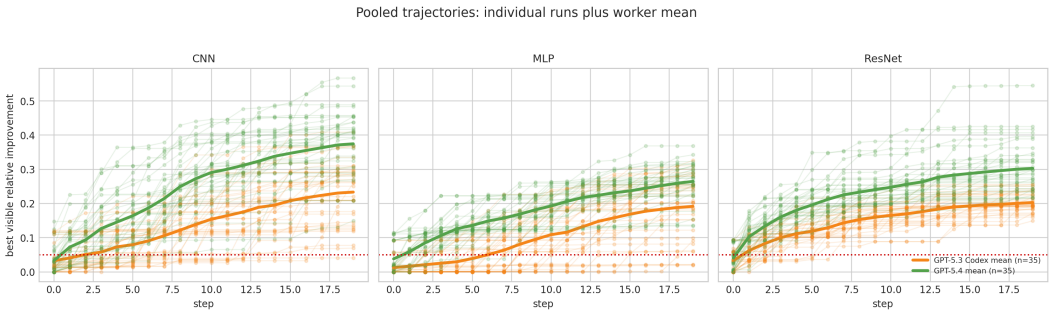

Figure 9: Best-so-far improvement trajectories. The mean trajectories show that `gpt_5_4` improves earlier and reaches higher final relative improvement in the pooled panel, but the deployment accounting in the main text shows that this does not automatically imply lower deployment loss after cost is charged. This contrast is the main empirical reason to separate worker competence from deployment value.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction state the paper's scope: verifiable agentic systems, a packed latent-mode assumption, decomposition-guided selection, and a controlled fixed-prototype experimental protocol. The discussion section separates theory-facing and deployment-facing claims.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 7 discusses the packed-mode assumption, small live-agent sample sizes, finite design menus, oracle mode labels, and current cost-measurement limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [Yes]

   Justification: Assumption 1 states the assumptions for the main theoretical result, Theorem 1 includes a proof sketch, and Appendix A gives the algebraic proof and pilot concentration details.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Section 5 describes the AutoResearch task instantiation, task modes, model menu, router-visible signals, horizons, metrics, and evaluation protocol; Appendix D specifies the required logs, estimators, and run-metadata fields.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [No]

   Justification: The paper is written against an existing repository and includes reproducibility commands, but an anonymized public code/data release is not included in the current submission package.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Section 5 describes the AutoResearch task instantiation, task modes, model menu, router-visible signals, primary estimators, horizons, and evaluation design. Additional implementation details are listed in Appendix D.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

Justification: Section 5 and Appendix D specify Wilson intervals, bootstrap intervals, survival curves, calibration diagnostics, and held-out evaluation for the reported protocol.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [No]

   Justification: The paper reports wall-clock costs and run counts but does not provide a complete compute-resource table for every experiment.

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work studies verifiable benchmark tasks and does not involve human subjects, sensitive personal data, or released high-risk models. The submission preserves anonymity and follows NeurIPS code and data policies.

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: Section 7 notes positive uses in cost-aware and transparent agent deployment as well as limitations and failure modes. The work is methodological and does not introduce a new generative model or dataset of people.

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper does not release a pre-trained model, image generator, scraped dataset, or other asset with high misuse risk. It evaluates agent harnesses on synthetic or generated benchmark tasks.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: The related-work section cites existing papers and systems used for positioning, and the experimental section describes the benchmark surfaces. Released code and benchmark assets include explicit license details.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The work introduces a code harness and analysis artifacts as research assets, and Appendix D documents the required logging and configuration fields.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

Justification: The paper does not involve crowdsourcing or research with human subjects.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The paper does not involve crowdsourcing or research with human subjects, so IRB or equivalent approval is not applicable.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [Yes]

    Justification: LLMs are central to the evaluated agent designs. Section 5 describes the three-worker menu, fixed prompt-only router, sequential signal protocol, model-identifier logging, and routing-output requirements.

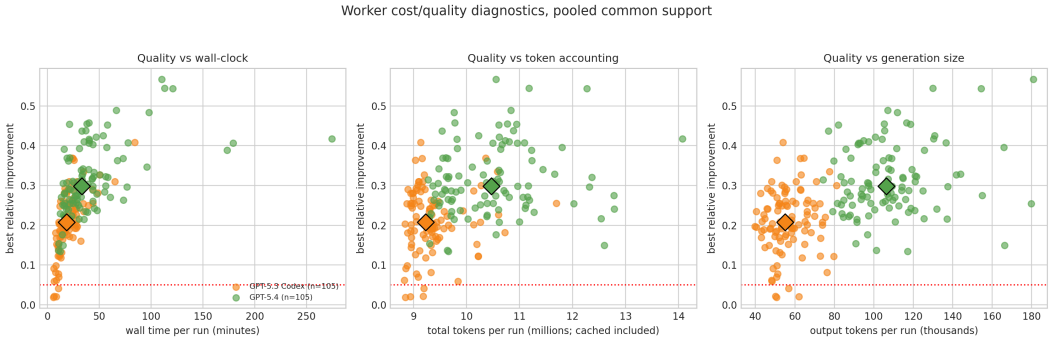

Figure 10: Worker cost–quality diagnostics. Each point is a pooled common-support run, and diamonds mark worker medians. The plots show that better final improvement is accompanied by substantial variation in wall-clock time, total token accounting, and generated-token volume. This supports the paper's budget-aware framing: worker choice cannot be reduced to final improvement alone.

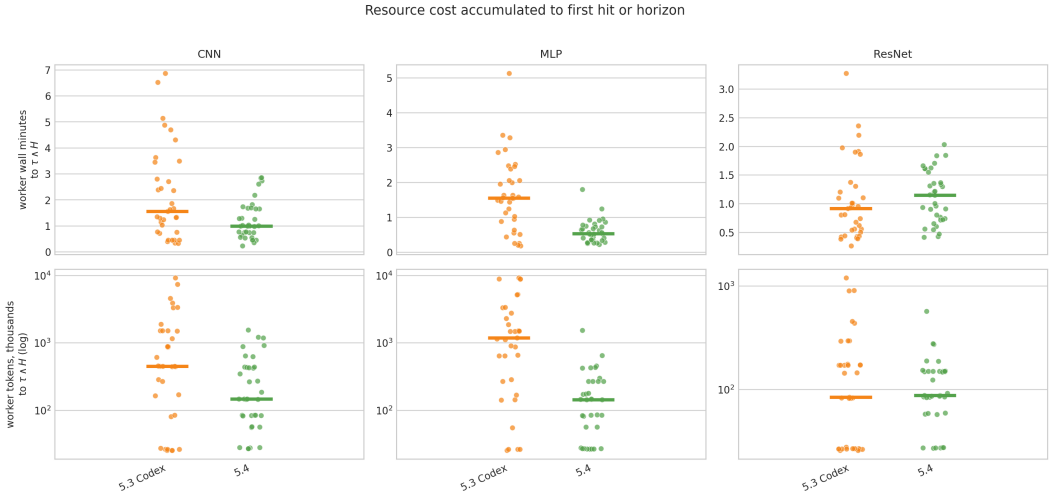

Figure 11: Resource cost accumulated to first hit or horizon. The top row reports wall-clock minutes and the bottom row reports token accounting on a log scale. Mode-conditional cost spreads are large, which is why the retry-crossover theorem is treated as a cost-sensitive design rule rather than a generic endorsement of weaker workers.

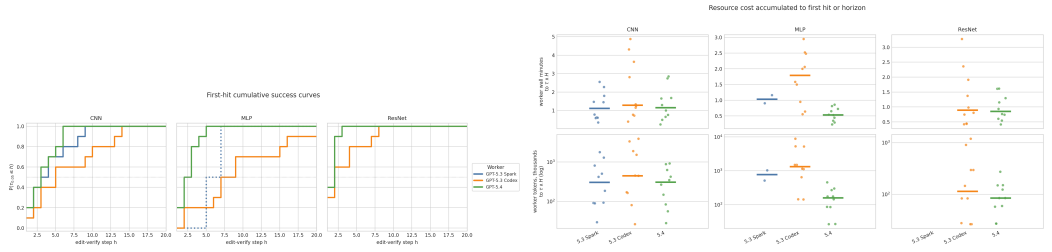

Figure 12: First-passage diagnostics. Left: first-hit CDFs by mode. Right: cost-to-threshold by mode and worker. These are the direct empirical objects behind the retry and time-to-certified-solution interpretation.

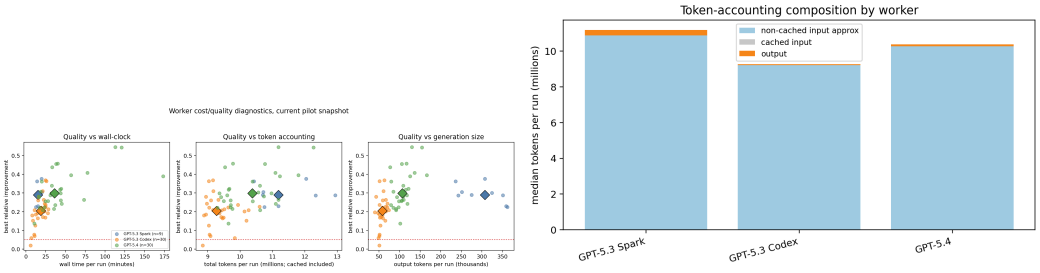

Figure 13: Worker-side cost and accounting diagnostics. These plots check whether conclusions are driven by wall-clock cost, token composition, or quality differences.

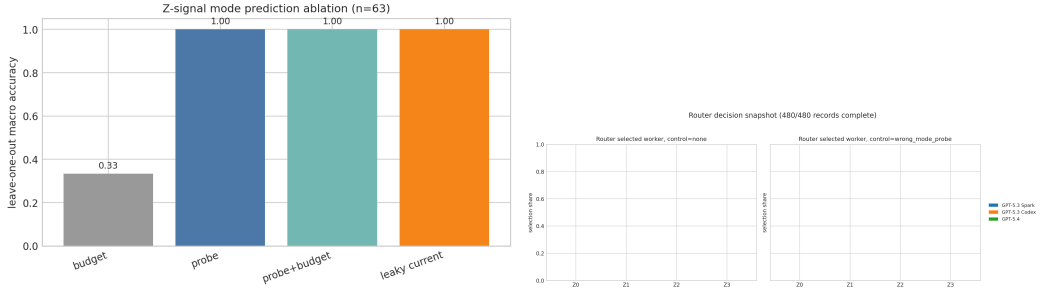

Figure 14: Router-facing diagnostics. The signal ablation tests whether progressively richer records expose task-mode structure; the router-selection snapshot checks whether the prompt-only router changes action distribution under those records.

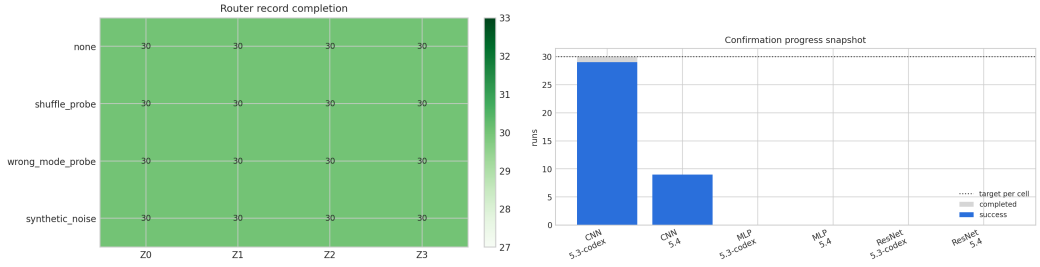

Figure 15: Run-coverage diagnostics for router records and holdout confirmation runs. These plots document which analyses are part of the promoted two-worker comparison and which are exploratory.